



CAHIER DES CHARGES TECHNIQUE

Projet : Cartes grises

Langages : MySQL – Java

Méthodologie : Agile Kanban (Trello)

Sommaire

1. Contexte du projet

1.1. Présentation du projet

1.2. Date de rendu du projet

2. Besoins fonctionnels

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

3.2. Ressources logicielles

4. Gestion du projet

5. Environnement collaboratif de développement logiciel

6. Conception du projet

6.1. Le front-end

6.1.1. Wireframes

6.1.2. Maquettes

6.1.3. Arborescences

7.2. Le back-end

7.2.1. Diagramme de cas d'utilisation

7.2.2. Diagramme d'activités

7.2.3. Diagramme de classe

7.2.4. Modèles Conceptuel de Données (MCD)

7.2.5. Modèle Logique de Données (MLD)

7.2.6. Modèle Physique de Données (MPD)

8. Technologies utilisées

8.1. Langages de développement Web

8.2. Base de données

9. Sécurité

9.1. Protection contre les injections SQL

1. Contexte du projet

1.1. Présentation du projet

La Direction des systèmes d'information de la préfecture vous a confié la mission de faire évoluer l'application métier du service des cartes grises. Cette application permet aux agents de gérer les informations liées aux véhicules, aux propriétaires ainsi qu'aux différentes opérations administratives.

Votre équipe et vous-même avez pour objectif d'analyser l'existant, de produire les documents de conception nécessaires afin de faciliter sa compréhension, puis de proposer une solution d'évolution répondant aux besoins du service.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 15/01/2025.re

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

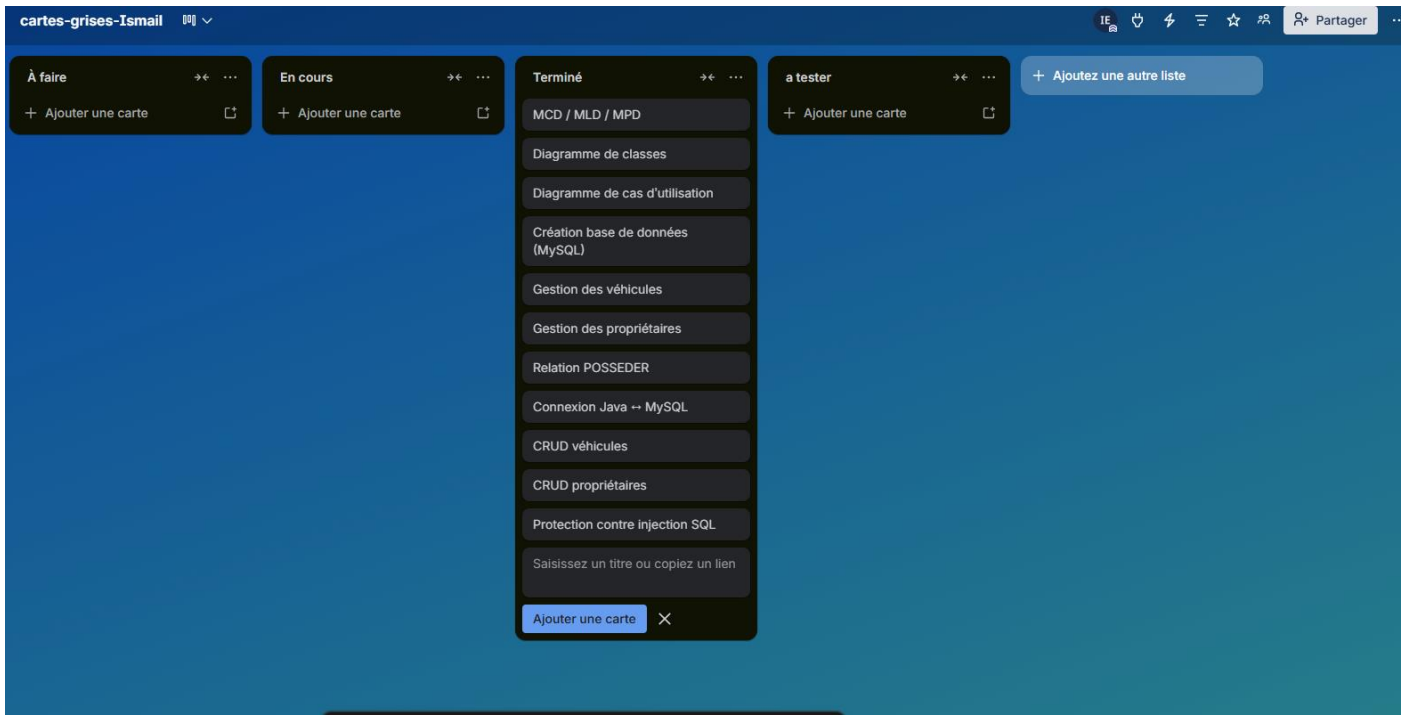
- PC
- Connexion internet
- Serveur local
- Périphérique de travail

3.2. Ressources logicielles

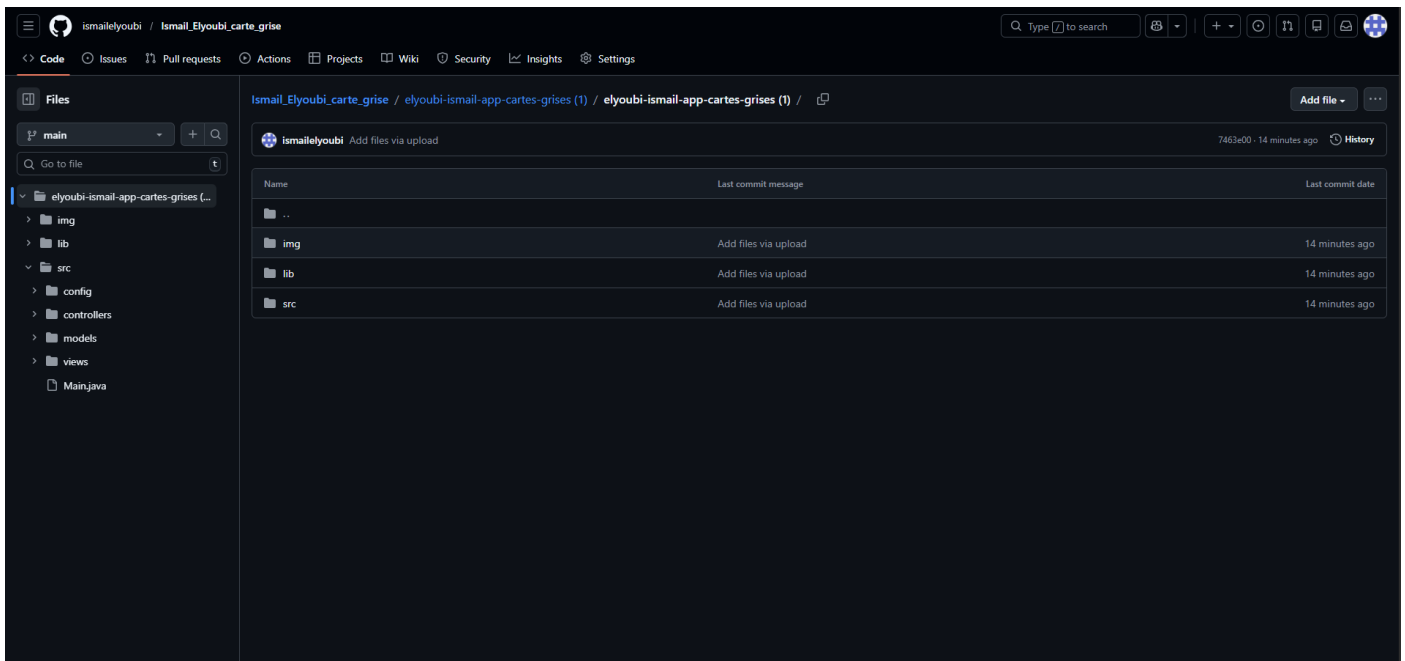
- Visual studio code
- Mysql
- Mamp
- Mocodo
- Java

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.



5. Environnement collaboratif de développement logiciel



6. Conception du projet

5.1. Le front-end

5.1.1. Wireframes

Bienvenue dans la gestion de carte grise

Gestion des Marques

Gestion des Modèles

Gestion des Véhicules

Gestion des Propriétaires

Gestion des Possessions

Propriétaire	Véhicule	Date début	Date fin	Modifier	Supprimer
John Doe	AB-123-CD	01/01/2023	31/12/2023	Modifier	Supprimer
Jane Doe	EF-456-GH	01/01/2023		Modifier	Supprimer
Alice Smith	IJ-789-KL	01/06/2023		Modifier	Supprimer

Ajouter une possession

Fermer

Propriétaire:

John Doe

Véhicule:

AB-123-CD

Date début (JJ/MM/AAAA):

Date fin (JJ/MM/AAAA):

Enregistrer

Propriétaire:

Alice Smith

Véhicule:

EF-456-GH

Date début (JJ/MM/AAAA):

16/12/2022

Date fin (JJ/MM/AAAA):

18/02/2023

Enregistrer

Nom :

Enregistrer

Nom :

Toyota

Enregistrer

ID	Nom	Modifier	Supprimer
1	Peugeot	Modifier	Supprimer
2	Renault	Modifier	Supprimer
3	Toyota	Modifier	Supprimer

Ajouter une Marque

Fermer

5.1.2. Maquettes

Bienvenue dans la gestion de carte grise

Gestion des Marques

Gestion des Modèles

Gestion des Véhicules

Gestion des Propriétaires

Gestion des Possessions

Liste des possessions

Propriétaire	Véhicule	Date début	Date fin	Modifier	Supprimer
John Doe	AB-123-CD	01/01/2023	31/12/2023	Modifier	Supprimer
Jane Doe	EF-456-GH	01/01/2023		Modifier	Supprimer
Alice Smith	IJ-789-KL	01/06/2023		Modifier	Supprimer

Ajouter une possession

Fermer

Formulaire possession

Propriétaire:

John Doe

Véhicule:

AB-123-CD

Date début (JJ/MM/AAAA):

Date fin (JJ/MM/AAAA):

Enregistrer

Formulaire possession

Propriétaire:

Alice Smith

Véhicule:

EF-456-GH

Date début (JJ/MM/AAAA):

16/12/2022

Date fin (JJ/MM/AAAA):

18/02/2023

Enregistrer

Formulaire marque

Nom :

Enregistrer

Formulaire marque

Nom :

Toyota

Enregistrer

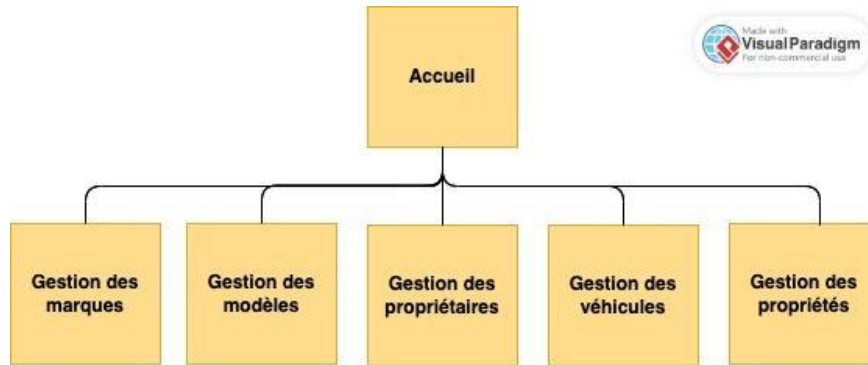
Liste des marques

ID	Nom	Modifier	Supprimer
1	Peugeot	Modifier	Supprimer
2	Renault	Modifier	Supprimer
3	Toyota	Modifier	Supprimer

Ajouter une marque

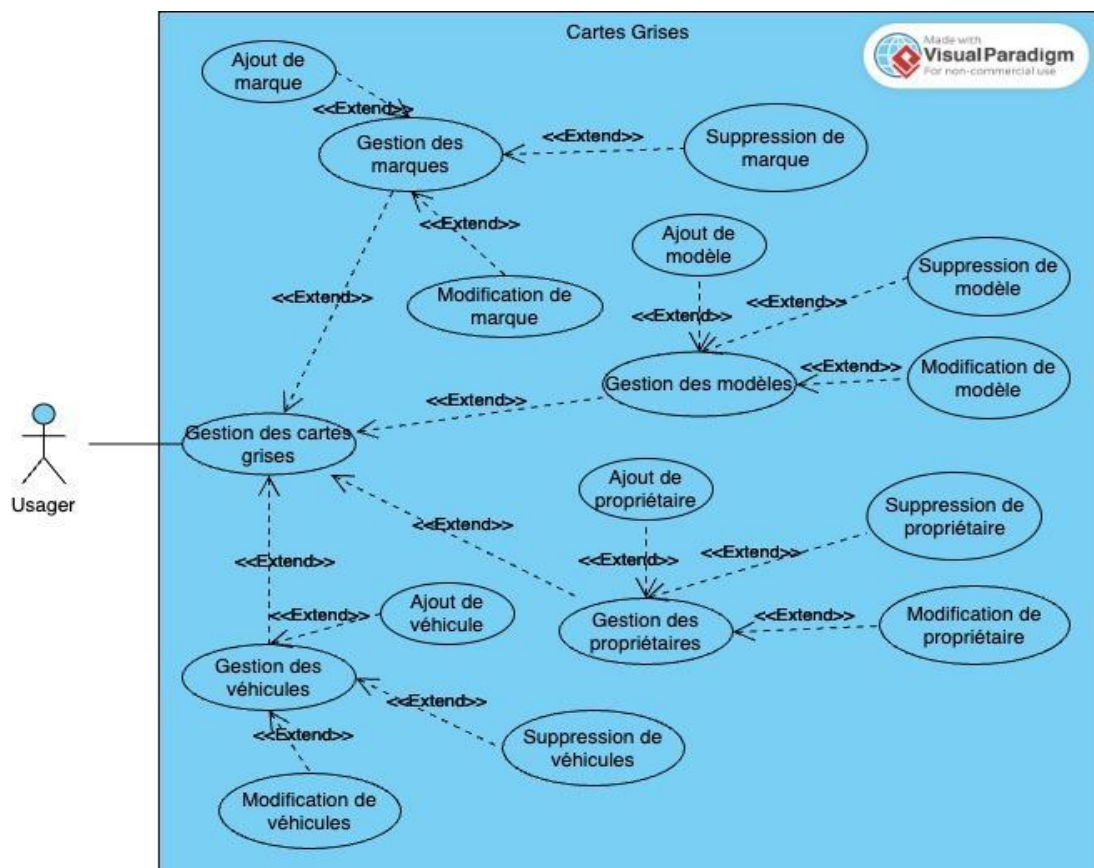
Fermer

5.1.3. Arborescences

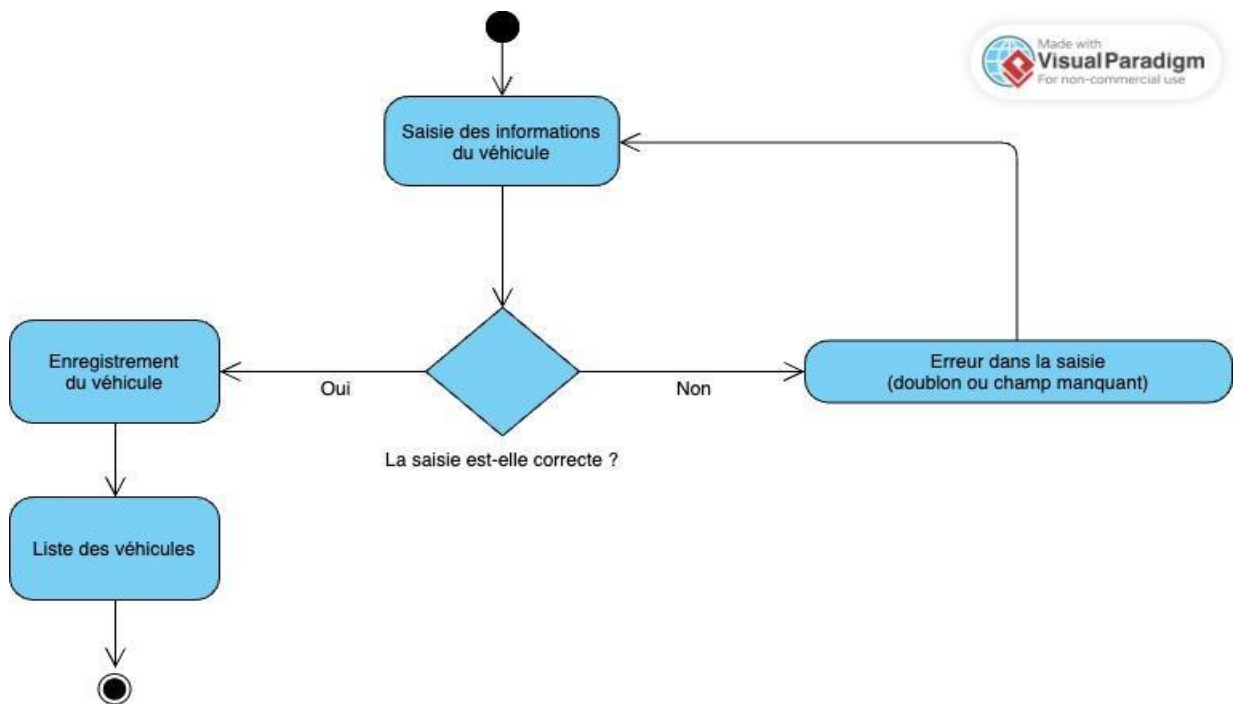


5.2. Le back-end

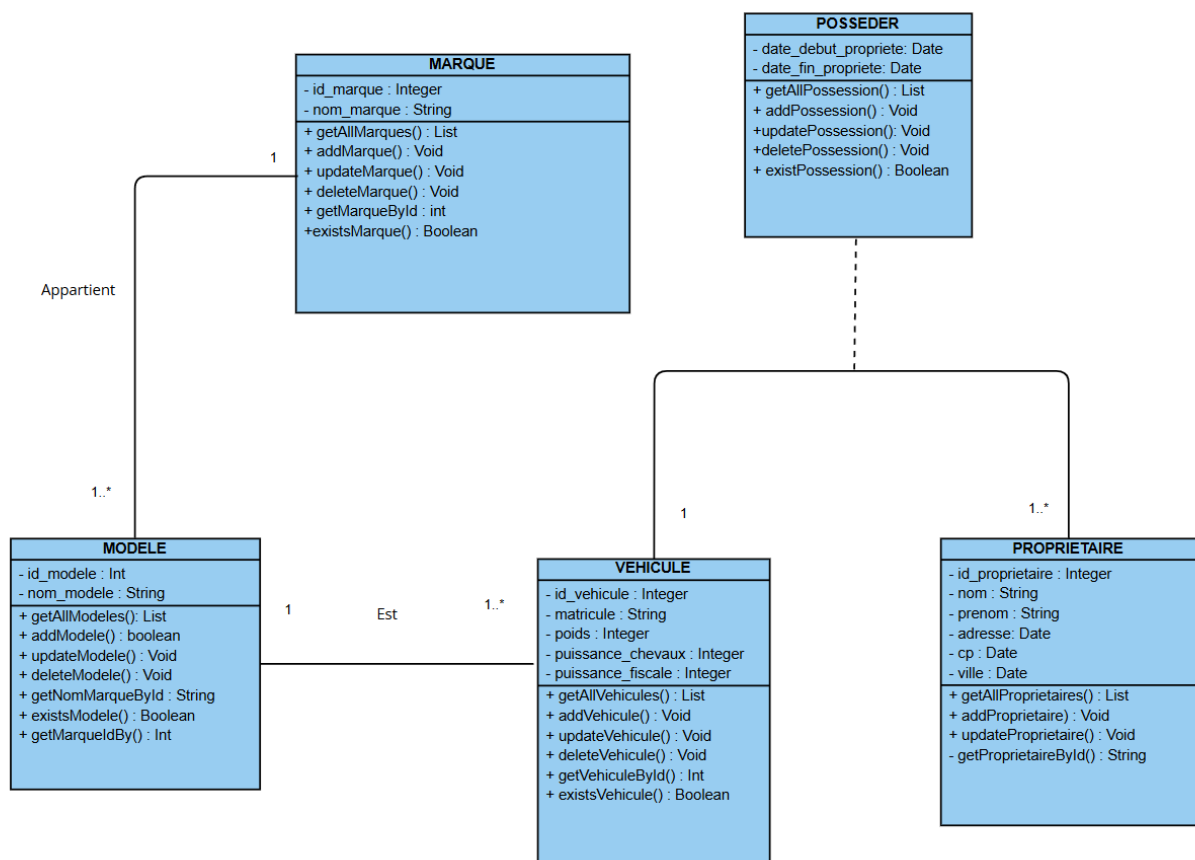
5.2.1. Diagramme de cas d'utilisation



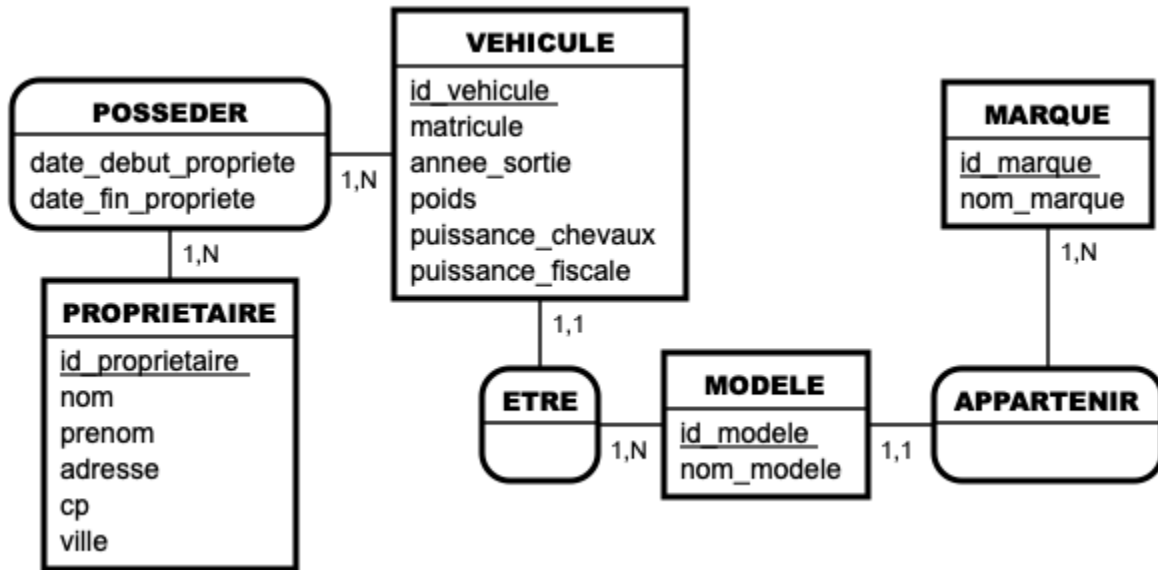
5.2.2. Diagramme d'activités



5.2.3. Diagramme de classe



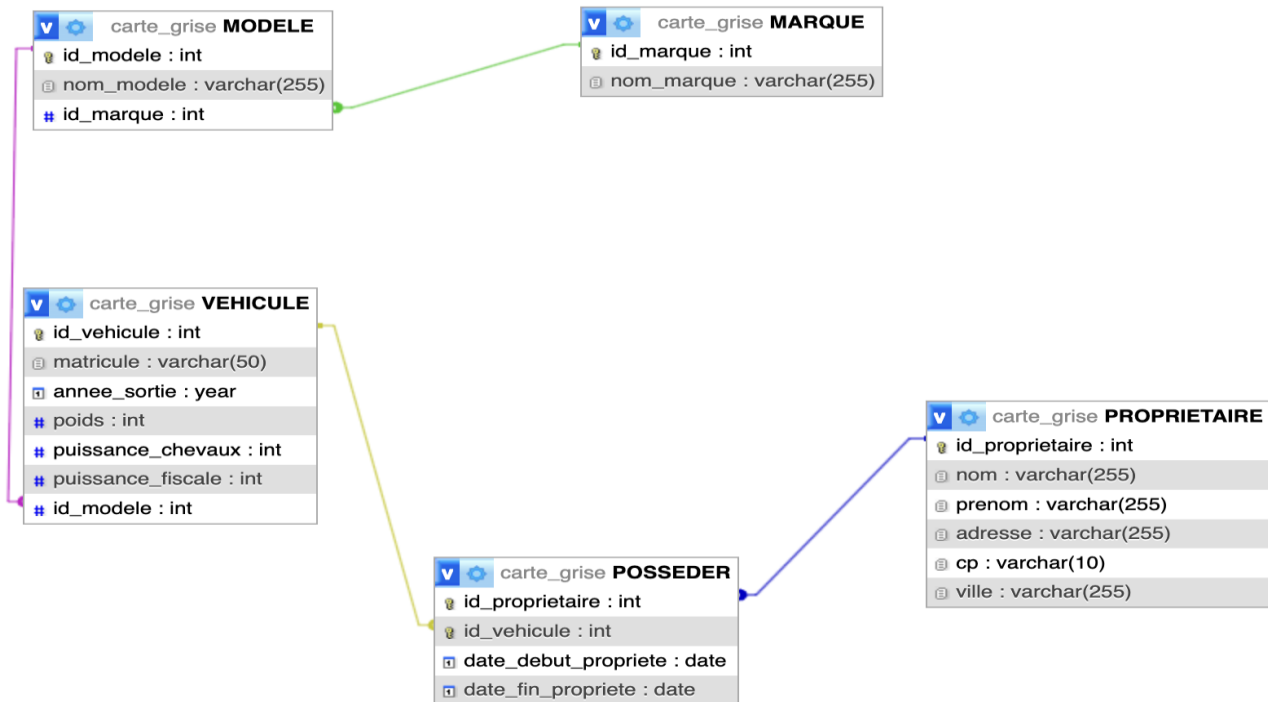
5.2.4. Modèles Conceptuel de Données (MCD)



5.2.5. Modèle Logique de Données (MLD)

- **MARQUE** (id_marque, nom_marque)
Clé primaire : id_marque
- **MODELE** (id_modele, nom_modele, id_marque)
Clé primaire : id_modele
Clé étrangère : id_marque en référence à id_marque de MARQUE
- **VEHICULE** (id_vehicule, matricule, annee_sortie, poids, puissance_chevaux, puissance_fiscale, id_modele)
Clé primaire : id_vehicule
Clé étrangère : id_modele en référence à id_modele de MODELE
- **PROPRIETAIRE** (id_proprietaire, nom, prenom, adresse, cp, ville)
Clé primaire : id_proprietaire
- **POSSEDER** (id_proprietaire, id_vehicule, date_debut_propriete, date_fin_propriete)
Clé primaire : id_proprietaire, id_vehicule
Clés étrangères : id_proprietaire en référence à id_proprietaire de PROPRIETAIRE id_vehicule en référence à id_vehicule de VEHICULE

5.2.6. Modèle Physique de Données (MPD)



7. Technologies utilisées

6.1. Langages de développement Web



6.2. Base de données



8. Sécurité

7.1. Protection contre les injections SQL

Définition de la solution :

Injection SQL : Attaque de sécurité qui permet à un utilisateur malveillant d'interférer avec les requêtes SQL qu'une application exécute sur sa base de données.

Comment ça marche ?

L'attaquant insère du code SQL malveillant dans des champs de saisie (formulaires, URL, etc.) qui est ensuite exécuté par la base de données lorsqu'il est concaténé sans protection dans une requête SQL.

Conséquences possibles :

- Accès non autorisé aux données
- Modification/suppression de données
- Élévation de privilèges
- Exécution de commandes système

Extraits de code

1. REQUÊTES PRÉPARÉES DANS Vehicule.java :

```
public static boolean existsPossession(int idProprietaire, int idVehicule) {
    String sql = "SELECT COUNT(*) FROM POSSEDER WHERE id_proprietaire=? AND id_vehicule=?";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, idProprietaire);
        ps.setInt(2, idVehicule);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getInt(1) > 0;
        }
    } catch (SQLException e) {
        System.err.println("Erreur existsPossession : " + e.getMessage());
    }
    return false;
}

}conditionnel

    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) return rs.getInt(1) > 0;
    }
    } catch (SQLException e) {
        System.err.println("Erreur exists : " + e.getMessage());
    }
    return false;
}
}
```

Explication :

La requête SQL contient deux ? qui sont remplacés par les valeurs entières passées en paramètre via ps.setInt(). Aucune concaténation de chaîne n'est utilisée, ce qui rend la requête **totalemt immunisée contre les injections SQL**, même si les paramètres venaient directement d'un formulaire utilisateur